Perimity — Complete Development Roadmap | CDAC Final Project

CDAC Mumbai | Final Major Project

Perimity

Smart Campus Access & Gate Pass Management System

Complete Development Roadmap — 25-Day Sprint Plan Database Design · APIs · UI Screens · Demo Script · Viva Guide

Tech Stack	Spring Boot 3.x (Java 17) │ React 18 │ PostgreSQL 16 │ MongoDB 7 │ RabbitMQ │ Redis │ AWS (EC2 + S3) │ Docker
Timeline	Days 1–5: Foundation │ Days 6–12: Core Backend │ Days 13–20: Frontend + Hardening │ Days 21–25: Deploy + Demo
Team Size	6 members │ 1 service each — every member owns backend **and** frontend for their service
Core Feature	Async RabbitMQ QR/PDF pipeline + email-keyed identity resolution for mixed-attendee bulk onboarding
API Testing	Swagger UI on every service — mandatory, this is how the team integrates

---

Part 1 — Technology Stack & Justifications

Every technology below is chosen for a specific reason. You will be asked "why this and not that" in your viva — knowing the *why* matters as much as the *what*.

Layer	Technology	Why Chosen
Backend	Spring Boot 3.x (Java 17)	Standard enterprise Java backend. Excellent JPA, Security, and messaging ecosystem.

Layer	Technology	Why Chosen
Frontend	React 18	Component reuse. Each team member builds their own page folder without collisions.
Microservices comms	REST over Docker Compose DNS	Containers resolve each other by service name — no service registry needed at this scale (see viva Q on Eureka).
Transactional DB	PostgreSQL 16	Strong consistency, real foreign keys, mature JSON support for `campus_config`.
Log/event store	MongoDB 7	Gate scans are write-heavy, append-only, join-free. Flat document per scan, shardable by `campus_id`.
Message broker	RabbitMQ	Decouples slow QR/PDF/email generation from the HTTP request. 600 passes generate in background; faculty never waits.
Cache	Redis	OTP attempt counters and session/token caching with native TTL expiry.
File storage	AWS S3	Photos, Aadhaar docs, QR PNGs, PDF passes. DB stores only the S3 key, never binary.
QR generation	ZXing (Java)	Google's open-source QR library. Encodes the signed token into a PNG in a few lines.
PDF generation	iText / OpenPDF	Composes the printable pass: QR image + name + photo + validity dates.
Auth	Passwordless email + OTP → JWT	No password to leak, forget, or reset. Visitors never create an account. JWT keeps services stateless.
Email	JavaMailSender (Gmail SMTP)	OTP delivery + welcome email with PDF pass attached. App Password, not main password.
Excel parsing	Apache POI	Faculty bulk-upload sheets (600-row event batches).
API docs/testing	springdoc-openapi (Swagger UI)	Every service self-documents. Teammates test each other's APIs in the browser before any UI exists.
Deployment	Docker + Docker Compose on AWS EC2	One command starts all 6 services + 4 infra containers. Same file locally and on EC2.

Layer	Technology	Why Chosen
CI	GitHub Actions	Auto-builds each service's Docker image on every push/PR.

Why PostgreSQL *and* MongoDB — why not just one?

Justification
Gate passes, users, and approvals need transactions, foreign keys, and strong consistency → PostgreSQL.
Gate scans are a firehose of append-only events with no joins — millions of rows over time → MongoDB.
A scan document is self-contained (pass id, guard, gate, timestamp, result). Forcing it into a relational table adds cost with no benefit.
MongoDB indexes `campus_id + scanned_at` as a compound index, covering nearly every query the organizer dashboard needs.
Using the right store per workload is a deliberate architectural decision — not indecision. Say exactly this in the viva.

Why RabbitMQ — the single most important justification

Justification
Problem: faculty uploads a 600-row event sheet. Generating 600 tokens + 600 QR PNGs + 600 PDFs + 600 emails synchronously would take minutes, time out the browser, and half would fail.
Fix: validate fast (~2 seconds), show "580 valid, 20 errors", faculty confirms, then drop 580 jobs into RabbitMQ and return immediately.
The QR Service consumes those jobs one at a time in the background. Faculty closes the laptop and goes home.
One failed job (bad email, S3 hiccup) does not block the other 579 — it is retried independently.
This is the difference between a demo that works with 5 rows and a system that works with 600.

Part 2 — Architecture & Service Ownership

2.1 Microservices Map

Service	Port	Database	Tech Highlights	Core Responsibility
auth-service	8081	AuthDB (PostgreSQL 5432)	Spring Security, JWT, Redis, JavaMailSender	OTP request/verify, JWT issuance, users, audit log
user-service	8082	UserDB (PostgreSQL 5433)	Spring JPA, S3 pre-signed URLs	Student/faculty profiles, departments, documents
gatepass-service	8083	GatePassDB (PostgreSQL 5434)	Apache POI, RabbitMQ producer	Visitor requests, pass lifecycle, events, bulk engine
campus-service	8084	CampusDB (PostgreSQL 5435)	Spring JPA	Campuses, gates, per-campus config
guard-service	8085	EntryLogDB (MongoDB 27017)	Spring Data MongoDB	QR scan endpoint — GREEN/RED/AMBER decision engine
qr-service	8086	QRDB (PostgreSQL 5436)	ZXing, iText, AWS SDK, RabbitMQ consumer	Token encryption, QR PNG, PDF, S3 upload

No API Gateway, no Eureka. Docker Compose puts all containers on one network and resolves them by service name (`http://qr-service:8080`). At one instance per service, a registry adds moving parts with no benefit. This is a defended decision, not an omission — see the viva section.

2.2 Ownership Model — Vertical Slices

There is **no frontend-only member**. Each of the 6 people owns:

1. Their Spring Boot service (entities → repository → service → controller → Swagger)
2. The React pages that call **only** their own service
3. Their own service's seed data and tests

Member	Service	Frontend folder
Member 1	auth-service	`frontend/src/auth/`
Member 2	user-service	`frontend/src/users/`
Member 3	gatepass-service	`frontend/src/gatepass/`
Member 4	campus-service	`frontend/src/campus/`
Member 5	guard-service	`frontend/src/guard/`
Member 6	qr-service	`frontend/src/qr/`

`frontend/src/shared/` (layout, routing, auth context) is touched by PR only.

2.3 Rules That Apply Every Single Day

Rule	Why
No `Semester` field in any UI	CDAC Mumbai has no semester concept. Departments are DAC, DBDA, DESD, DITISS, DMLT, PGAIML.
Passwordless everywhere — email + OTP	Project decision. Even admins. No password field exists anywhere.
Files to S3, keys to DB	Never store binary in a database.
Passwords bcrypt, OTP SHA-256, QR token AES-256	No secret is ever stored in plain text.
No service reads another service's database	Cross-service data comes from an API call, always.
Every endpoint gets a Swagger `@Operation` summary	An endpoint without Swagger docs is not "done".

Part 3 — Database Design

Database-per-service. Six databases, no shared tables.

3.1 AuthDB (PostgreSQL, port 5432) — owned by auth-service

Table: users

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
email	VARCHAR(150)	UNIQUE, NOT NULL	The universal key across the whole system
name	VARCHAR(100)	NOT NULL	
phone	VARCHAR(15)	NULLABLE	
password_hash	VARCHAR(255)	NULLABLE	bcrypt. NULL for OTP-only users (most users)
role	VARCHAR(20)	NOT NULL	STUDENT, FACULTY, VISITOR, ADMIN, GUARD
campus_id	BIGINT	NOT NULL	Multi-tenant scope
is_active	BOOLEAN	DEFAULT TRUE	
created_at	TIMESTAMP	DEFAULT NOW()	

Table: otp_verifications

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
email	VARCHAR(150)	NOT NULL, INDEX	
otp_hash	VARCHAR(64)	NOT NULL	SHA-256. Never the plain OTP
expires_at	TIMESTAMP	NOT NULL	Created time + 10 minutes
attempts	INT	DEFAULT 0	Locked at 5
is_used	BOOLEAN	DEFAULT FALSE	Single use
created_at	TIMESTAMP	DEFAULT NOW()	

Table: audit_logs

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
user_id	BIGINT	NULLABLE	NULL for failed logins with unknown email
action	VARCHAR(50)	NOT NULL	LOGIN, OTP_FAILED, PASS_APPROVED, USER_DEACTIVATED
entity_type	VARCHAR(50)	NULLABLE	
entity_id	BIGINT	NULLABLE	
ip_address	VARCHAR(45)	NULLABLE	IPv6-safe length
campus_id	BIGINT	NOT NULL	
created_at	TIMESTAMP	DEFAULT NOW()	

3.2 UserDB (PostgreSQL, port 5433) — owned by user-service

Table: student_profiles

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
user_id	BIGINT	UNIQUE, NOT NULL	Reference to AuthDB users.id (by convention, not FK)
roll_no	VARCHAR(30)	UNIQUE, NOT NULL	
department_id	BIGINT	NOT NULL	FK → departments.id
year	INT	NULLABLE	**No semester column — deliberate**
aadhaar_number	VARCHAR(12)	NULLABLE	
address	TEXT	NULLABLE	
photo_s3_key	VARCHAR(255)	NULLABLE	S3 key only, never the image

Column	Type	Constraint	Notes
campus_id	BIGINT	NOT NULL	

Table: faculty_profiles

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
user_id	BIGINT	UNIQUE, NOT NULL	
employee_id	VARCHAR(30)	UNIQUE, NOT NULL	
designation	VARCHAR(100)	NULLABLE	
qualification	VARCHAR(150)	NULLABLE	
department_id	BIGINT	NULLABLE	
photo_s3_key	VARCHAR(255)	NULLABLE	
campus_id	BIGINT	NOT NULL	

Table: departments

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
code	VARCHAR(20)	NOT NULL	DAC, DBDA, DESD, DITISS, DMLT, PGAIML
name	VARCHAR(100)	NOT NULL	
campus_id	BIGINT	NOT NULL	Each campus has its own list

Table: documents

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
user_id	BIGINT	NOT NULL	

Column	Type	Constraint	Notes
doc_type	VARCHAR(50)	NOT NULL	AADHAAR, CERTIFICATE, PHOTO
s3_key	VARCHAR(255)	NOT NULL	
file_name	VARCHAR(200)	NOT NULL	
mime_type	VARCHAR(100)	NOT NULL	
is_verified	BOOLEAN	DEFAULT FALSE	
uploaded_at	TIMESTAMP	DEFAULT NOW()	

3.3 GatePassDB (PostgreSQL, port 5434) — owned by gatepass-service

This is the core business logic database.

Table: visitor_requests

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
visitor_name	VARCHAR(100)	NOT NULL	
visitor_email	VARCHAR(150)	NOT NULL	
visitor_phone	VARCHAR(15)	NULLABLE	
purpose	TEXT	NOT NULL	
host_user_id	BIGINT	NULLABLE	Who they’re visiting
visit_from	DATE	NOT NULL	
visit_to	DATE	NOT NULL	
otp_verified	BOOLEAN	DEFAULT FALSE	Must be TRUE before approval
status	VARCHAR(20)	DEFAULT ‘PENDING’	PENDING, APPROVED, REJECTED
approved_by	BIGINT	NULLABLE	
campus_id	BIGINT	NOT NULL	

Column	Type	Constraint	Notes
created_at	TIMESTAMP	DEFAULT NOW()	

Table: gate_passes

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
holder_user_id	BIGINT	NOT NULL	The identity holding this pass
holder_name	VARCHAR(100)	NOT NULL	Denormalized for fast scan display
pass_type	VARCHAR(10)	NOT NULL	**DAILY** or **EVENT**
event_id	BIGINT	NULLABLE	NULL for DAILY, set for EVENT
valid_from	DATE	NOT NULL	
valid_to	DATE	NULLABLE	**NULL for DAILY passes — no end date**
status	VARCHAR(20)	DEFAULT ‘PENDING’	PENDING → ACTIVE → EXPIRED / REVOKED
qr_s3_key	VARCHAR(255)	NULLABLE	Filled by QR Service when generated
pdf_s3_key	VARCHAR(255)	NULLABLE	Filled by QR Service when generated
campus_id	BIGINT	NOT NULL	
created_at	TIMESTAMP	DEFAULT NOW()	

Table: events

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
campus_id	BIGINT	NOT NULL	
name	VARCHAR(150)	NOT NULL	e.g. “AI Summit”
valid_from	DATE	NOT NULL	Event start
valid_to	DATE	NOT NULL	Event end

Column	Type	Constraint	Notes
created_by	BIGINT	NOT NULL	Faculty/admin user id
created_at	TIMESTAMP	DEFAULT NOW()	

Table: bulk_upload_batches

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
uploaded_by	BIGINT	NOT NULL	
batch_type	VARCHAR(10)	NOT NULL	STUDENT or VISITOR
event_id	BIGINT	NULLABLE	Set when batch_type = VISITOR
file_s3_key	VARCHAR(255)	NOT NULL	The uploaded Excel
total_rows	INT	DEFAULT 0	
valid_rows	INT	DEFAULT 0	
invalid_rows	INT	DEFAULT 0	
processed_rows	INT	DEFAULT 0	Increments as RabbitMQ jobs complete
error_report_s3_key	VARCHAR(255)	NULLABLE	Downloadable “row 34: invalid email” report
status	VARCHAR(20)	DEFAULT ‘VALIDATING’	VALIDATING → AWAITING_CONFIRM → PROCESSING → DONE
campus_id	BIGINT	NOT NULL	
created_at	TIMESTAMP	DEFAULT NOW()	

3.4 CampusDB (PostgreSQL, port 5435) — owned by campus-service

Table: campuses

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
name	VARCHAR(150)	NOT NULL	CDAC Mumbai, CDAC Pune
code	VARCHAR(20)	UNIQUE, NOT NULL	
address	TEXT	NULLABLE	
logo_s3_key	VARCHAR(255)	NULLABLE	
admin_user_id	BIGINT	NULLABLE	
is_active	BOOLEAN	DEFAULT TRUE	

Table: campus_gates

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
campus_id	BIGINT	NOT NULL	
name	VARCHAR(50)	NOT NULL	Main Gate, Back Gate
location	VARCHAR(100)	NULLABLE	
is_active	BOOLEAN	DEFAULT TRUE	

Table: campus_config

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
campus_id	BIGINT	NOT NULL	
config_key	VARCHAR(50)	NOT NULL	approval_required, reentry_allowed
config_value	VARCHAR(255)	NOT NULL	Key-value = new settings without schema changes

3.5 EntryLogDB (MongoDB, port 27017) — owned by guard-service

Collection: entry_logs

Field	Type	Notes
_id	ObjectId	
pass_id	Long	The pass that was scanned
holder_user_id	Long	
holder_name	String	Denormalized for instant display
event_id	Long / null	Set if attributed to an event (including Behavior 2 auto-attribution)
scan_result	String	GREEN, RED, AMBER
deny_reason	String / null	EXPIRED, REVOKED, INVALID_TOKEN, NOT_YET_VALID
gate_id	Long	
guard_id	Long	
campus_id	Long	
scanned_at	ISODate	
device_info	String / null	

Indexes

Index	Type	Covers
campus_id + scanned_at	Compound	Nearly every dashboard and log query
pass_id	Single	Scan history for one pass
holder_user_id	Single	Person’s entry history
event_id + scanned_at	Compound	Organizer attendance per day

3.6 QRDB (PostgreSQL, port 5436) — owned by qr-service

Table: qr_records

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
pass_id	BIGINT	UNIQUE, NOT NULL	
token_hash	VARCHAR(64)	UNIQUE, NOT NULL	SHA-256 of the token, for lookup
encrypted_token	TEXT	NOT NULL	AES-256(pass_id + campus_id + expiry)
qr_s3_key	VARCHAR(255)	NOT NULL	
pdf_s3_key	VARCHAR(255)	NOT NULL	
valid_from	DATE	NOT NULL	
valid_to	DATE	NULLABLE	
is_active	BOOLEAN	DEFAULT TRUE	Set FALSE on revoke
created_at	TIMESTAMP	DEFAULT NOW()	

Table: generation_jobs

Column	Type	Constraint	Notes
id	BIGSERIAL	PK	
pass_id	BIGINT	NOT NULL	
batch_id	BIGINT	NULLABLE	Links back to a bulk upload
status	VARCHAR(20)	DEFAULT ‘QUEUED’	QUEUED → PROCESSING → DONE / FAILED
retry_count	INT	DEFAULT 0	Bounded retries, then manual escalation
error_message	TEXT	NULLABLE	
created_at	TIMESTAMP	DEFAULT NOW()	
completed_at	TIMESTAMP	NULLABLE	

3.7 S3 Folder Structure

```
perimity-bucket/
├── campuses/
│   └── campus-1/logo.png
├── profiles/
│   └── user-42/
│       ├── photo.jpg
│       └── aadhaar.pdf
├── passes/
│   └── campus-1/
│       ├── pass-123-qr.png
│       └── pass-123.pdf
└── bulk/
    └── batch-9/
        ├── upload.xlsx
        └── error-report.xlsx
```

Part 4 — REST API Design

Every endpoint below must have a Swagger `@Operation(summary = "...")` annotation.

4.1 auth-service (port 8081)

M	Endpoint	Purpose	Auth
POST	/api/auth/otp/request	Body `{email}`. Generate OTP, SHA-256 hash it, store, email it.	None
POST	/api/auth/otp/verify	Body `{email, otp}`. Validate → return JWT + user profile.	None
GET	/api/auth/me	Current logged-in user.	Any
POST	/api/auth/logout	Invalidate session/token in Redis.	Any
POST	/api/auth/internal/users	Create identity. Called by user-service and gatepass-service (internal API key header).	Internal

M	Endpoint	Purpose	Auth
GET	/api/auth/internal/users/by-email	Does this email already exist? Used by the bulk engine.	Internal
GET	/api/auth/audit-logs	Paginated audit trail. Query: `?action=LOGIN&from=&to=`	ADMIN

4.2 user-service (port 8082)

M	Endpoint	Purpose	Auth
POST	/api/users/students	Create student profile (calls auth-service to create identity first).	ADMIN
GET	/api/users/students/{id}	Student profile detail.	Any (own) / ADMIN
PUT	/api/users/students/{id}	Update profile. **No semester field.**	Own / ADMIN
GET	/api/users/students	List/search students. Query: `?department=DAC&page=0`	ADMIN, FACULTY
POST	/api/users/faculty	Create faculty profile.	ADMIN
GET	/api/users/faculty/{id}	Faculty detail.	Own / ADMIN
GET	/api/users/departments	List departments for the campus.	Any
POST	/api/users/documents/presign	Get an S3 pre-signed upload URL.	Any
POST	/api/users/documents	Register uploaded doc's S3 key + metadata.	Any
PUT	/api/users/documents/{id}/verify	Mark document verified.	ADMIN

4.3 gatepass-service (port 8083)

M	Endpoint	Purpose	Auth
POST	/api/gatepass/visitor-requests	Visitor submits registration form.	None (OTP-gated)
GET	/api/gatepass/visitor-requests	Pending requests queue.	ADMIN, FACULTY
PUT	/api/gatepass/visitor-requests/{id}/approve	Approve → create pass → publish RabbitMQ job.	ADMIN, FACULTY
PUT	/api/gatepass/visitor-requests/{id}/reject	Reject with reason.	ADMIN, FACULTY
GET	/api/gatepass/passes/me	My active pass(es) — may return both a DAILY and an EVENT pass.	Any
GET	/api/gatepass/passes/{id}	Pass detail.	Own / ADMIN
DELETE	/api/gatepass/passes/{id}	Revoke pass (also deactivates the QR record).	ADMIN
PUT	/api/gatepass/internal/passes/{id}/activate	Called by qr-service when QR+PDF are ready.	Internal
GET	/api/gatepass/internal/passes/{userId}/active-event	Does this person have an event running today? Used by guard-service for Behavior 2.	Internal
POST	/api/gatepass/events	Create event (name, campus, date range).	ADMIN, FACULTY
GET	/api/gatepass/events	List events.	ADMIN, FACULTY
GET	/api/gatepass/events/{id}/attendance	Registered / attended per day / never showed.	ADMIN, FACULTY

M	Endpoint	Purpose	Auth
GET	/api/gatepass/events/{id}/attendance/export	CSV download.	ADMIN, FACULTY
POST	/api/gatepass/bulk-upload	Upload Excel. Returns batchId + validation summary.	ADMIN, FACULTY
POST	/api/gatepass/bulk-upload/{batchId}/confirm	Confirm → create identities + publish N RabbitMQ jobs.	ADMIN, FACULTY
GET	/api/gatepass/bulk-upload/{batchId}/status	Poll processing progress.	ADMIN, FACULTY
GET	/api/gatepass/bulk-upload/{batchId}/errors	Download error report.	ADMIN, FACULTY

4.4 campus-service (port 8084)

M	Endpoint	Purpose	Auth
GET	/api/campus	List campuses.	Any
POST	/api/campus	Create campus.	ADMIN
GET	/api/campus/{id}	Campus detail + stats.	ADMIN
PUT	/api/campus/{id}	Update campus.	ADMIN
GET	/api/campus/{id}/gates	List gates.	Any
POST	/api/campus/{id}/gates	Add gate.	ADMIN
PUT	/api/campus/gates/{gateId}	Edit/deactivate gate.	ADMIN
GET	/api/campus/{id}/config	All config key-values.	ADMIN
PUT	/api/campus/{id}/config	Upsert a config key.	ADMIN
GET	/api/campus/internal/{id}/config/{key}	Single config lookup for other services.	Internal

4.5 guard-service (port 8085)

M	Endpoint	Purpose	Auth
POST	/api/guard/scan	Body `{token, gateId}`. **The core endpoint.** Returns `{result, message, holderName, eventName}`.	GUARD
GET	/api/guard/logs	Scan log. Query: `?date=&gateId=&result=RED`	GUARD, ADMIN
GET	/api/guard/logs/pass/{passId}	Full scan history for one pass.	ADMIN
GET	/api/guard/internal/attendance/{eventId}	Per-day attendance counts. Consumed by gatepass-service's dashboard.	Internal

4.6 qr-service (port 8086)

M	Endpoint	Purpose	Auth
GET	/api/qr/{passId}	QR + PDF S3 keys and validity.	Own / ADMIN
GET	/api/qr/{passId}/download	Stream/redirect the PDF pass.	Own / ADMIN
GET	/api/qr/jobs/{jobId}/status	Async job status.	ADMIN, FACULTY
GET	/api/qr/jobs/batch/{batchId}/progress	"412 of 580 generated" for the bulk progress bar.	ADMIN, FACULTY
POST	/api/qr/internal/decrypt	Decrypt a scanned token → pass details. Called by guard-service.	Internal
PUT	/api/qr/internal/{passId}/deactivate	Called when a pass is revoked.	Internal

Part 5 — UI Screens (16 Screens)

#	Screen	Owner	Who Sees It	Key Components	User Flow
1	Email Entry	auth	All	Single email field, “Continue” button	Enter email → OTP sent → Screen 2
2	OTP Verify	auth	All	6-digit boxes, 10-min countdown, resend link, attempts-left warning	Enter OTP → JWT stored → role-based redirect
3	Student Profile	user	STUDENT, ADMIN	Photo, roll no, department, year, address, Aadhaar upload. **No semester field**	View → edit → save → toast
4	Faculty Profile	user	FACULTY, ADMIN	Employee ID, designation, qualification, photo	View → edit → save
5	Student Directory	user	ADMIN, FACULTY	Searchable table, department filter chips, pagination	Search → click row → profile
6	Visitor Registration	gatepass	Public	Name, email, phone, purpose, host, date range → OTP step → confirmation	Fill → verify OTP → “request submitted”
7	Approval Queue	gatepass	ADMIN, FACULTY	Pending request cards, Approve/Reject buttons, reject-reason modal	Review → approve → pass generating toast
8	My Pass	gatepass	Any holder	Pass card(s) — may show **both** a DAILY and an EVENT pass side by side	View → tap → Screen 14 for download
9	Bulk Upload	gatepass	ADMIN, FACULTY	Drag-drop Excel, type selector (student/visitor), event picker, **“580 valid, 20 errors”** summary panel, Confirm button, error report download	Upload → review summary → confirm → “passes generating”

#	Screen	Owner	Who Sees It	Key Components	User Flow
10	Bulk Progress	gatepass/qr	ADMIN, FACULTY	Live progress bar polling job status: “412 of 580 generated”	Watch or leave — background job continues
11	Event Management	gatepass	ADMIN, FACULTY	Event list, create form (name + date range)	Create event → use it in Screen 9
12	Attendance Dashboard	gatepass	ADMIN, FACULTY	Registered / Attended Day 1 / Day 2 / Never showed cards, per-day bar chart, attendee search, **Export CSV**	The payoff screen — a paper register can never do this
13	Guard Scanner	guard	GUARD	Camera QR scanner + manual token fallback, then **full-screen GREEN / RED / AMBER result card** with holder name and message, auto-reset after 5s	Scan → instant colour → next visitor
14	Pass Download	qr	Any holder	Large QR image, validity dates, Download PDF button	Open → show QR at gate or download
15	Guard Log	guard	GUARD, ADMIN	Table: name, time, gate, result badge, deny reason. Date + result filters	Filter RED to review denied attempts
16	Campus Admin	campus	ADMIN	Campus list/edit, gate management table, config toggles (approval required, re-entry allowed)	Add gate → toggle config → save

Shared components (in `frontend/src/shared/`, PR-only): Navbar, Sidebar (role-based links), ProtectedRoute, Toast, LoadingSpinner, EmptyState, AuthContext.

Part 6 — The Core Technical Pieces

These three are your "we actually engineered something" talking points. Know them cold.

6.1 The Async QR/PDF Pipeline

Step	What happens
1	Admin approves a visitor request (or bulk batch is confirmed).
2	gatepass-service INSERTs `gate_passes` row with `status = PENDING`, **no QR yet**.
3	gatepass-service publishes to RabbitMQ queue `pass.generate`: `{pass_id, holder_name, campus_id, valid_from, valid_to}`. HTTP response returns **immediately**.
4	qr-service `@RabbitListener` consumes the job, writes a `generation_jobs` row with `status = PROCESSING`.
5	Generates an AES-256 signed token (pass_id + campus_id + expiry).
6	ZXing encodes the token into a 300×300 QR PNG.
7	iText composes the PDF pass: QR + holder name + photo + validity dates.
8	AWS SDK uploads both to S3 → `passes/campus-1/pass-123-qr.png`, `passes/campus-1/pass-123.pdf`.
9	Stores `token_hash` + both S3 keys in `qr_records`. Job → DONE.
10	Calls `PUT /api/gatepass/internal/passes/{id}/activate` → gatepass-service sets `status = ACTIVE` and stores the S3 keys.
11	Publishes to `notification.send` → welcome email with the PDF attached.

Failure handling: a failed job increments `retry_count` and is re-queued up to 3 times, then marked FAILED with the error message for manual escalation. One failure never blocks the other 579.

6.2 Mixed-Attendee Resolution — Email as the Universal Key

The hardest business logic in the project. A faculty uploads 600 rows for "AI Summit". 102 of those people are already CDAC students. The faculty does **not** know which ones.

```
For each Excel row (matched by email):
  ├─ Email exists as student/faculty of ANY campus?
  │     → REUSE existing identity
  │     → issue EVENT pass only
  │     → do NOT create a duplicate account
  │
  └─ Email is brand new?
        → create lightweight VISITOR identity (name, email, phone only —
          no roll number, no department)
        → issue EVENT pass
```

Case	Result
CDAC Mumbai student attending	Existing identity reused. Now holds DAILY + EVENT pass simultaneously.
CDAC Pune student attending Mumbai's event	Recognized by email. Identity reused. Event pass scoped to Mumbai's event.
Outside attendee	New lightweight visitor identity + event pass. After the event, pass expires; identity remains for audit ("who attended AI Summit 2026").
Same email twice in the sheet	Skipped, flagged in error report.
Email on campus blocklist	Skipped automatically, flagged in report.

All 600 receive an event pass. Only the genuinely new people get a new identity. **Zero duplicates.**

6.3 The Two-QR Problem and Gate Scan Decision Tree

A student attending the event holds two valid QRs — their DAILY pass and their EVENT pass. Out of habit they scan the daily one. Naively, the organizer's attendance list would be wrong.

Behavior 2 solves it: if the scanned pass is DAILY but that person has an event running today, auto-attribute the entry to that event. The guard sees one green light. The organizer gets accurate attendance. The system does the attribution — not the guard.

```
Guard scans QR
   ↓
qr-service decrypts token → pass_id, campus_id, expiry
   ↓
Pass valid + active + in date range?
   ├─ NO  → RED + reason (EXPIRED / REVOKED / INVALID_TOKEN / NOT_YET_VALID)
```

```
│        → still logged to entry_logs (denied attempts matter for security)
└─ YES
     ↓
  Is this an EVENT pass?
  ├─ YES → log with event_id → GREEN "Welcome to [Event]"
  └─ NO (DAILY pass)
       ↓
     Does this person have an event running today?  ← Behavior 2
     ├─ YES → log with that event_id → GREEN "Welcome to [Event]"
     └─ NO  → log normal entry, event_id = null → GREEN "Welcome"
```

Rule	Detail
Entry-only	No exit scan, no in/out toggle. This mirrors the paper register it replaces.
Repeat entries	All logged. A person may enter several times a day — each is a row, exactly like multiple lines in a register.
Multi-day events	Each day's first scan is that day's attendance. A 3-day event = up to 3 entry logs per person, grouped by day.
AMBER	Reserved for "needs attention but not a hard deny" — e.g. pass expires today, or campus config requires manual host confirmation.

Part 7 — Day-by-Day Plan (25 Days)

Daily ritual: 15-minute standup, same time every day — what I finished, what I'm doing, what's blocking me.

Week 1 — Foundation (Days 1–5)

Day 1 — Environment + Repo + Skeletons
INSTALL: Everyone installs JDK 17, Docker Desktop, Node.js 20, and an IDE (STS / IntelliJ / VS Code).
REPO: Clone the org repo. Confirm `docker-compose up -d` starts postgres, mongo, rabbitmq, redis. Verify RabbitMQ UI at localhost:15672 (guest/guest).
OWNERSHIP: Lock in who owns which service. Replace "Owner: TBD" in each service README with a real name. Commit it.

Day 1 — Environment + Repo + Skeletons
SKELETON: Each owner generates a Spring Boot project via Spring Initializr into their folder. Dependencies: Web, JPA (or Spring Data MongoDB for guard), Validation, Lombok, springdoc-openapi.
SWAGGER: Add `springdoc-openapi-starter-webmvc-ui` 2.6.0. Confirm Swagger UI loads at `localhost:<port>/swagger-ui.html`.
PING: Each owner writes `GET /api/<service>/ping` returning `{"status":"ok"}`, with an `@Operation` summary.
BRANCH: Each owner pushes on `feature/<service>-setup` and opens a PR. Practice the review-and-merge flow once, today.
DELIVERABLE: All 6 skeletons boot. All 6 Swagger pages load. All 6 first PRs merged to main.

Day 2 — Data Modeling (all services in parallel)
ENTITIES: Each owner writes their JPA entities (or Mongo documents) exactly matching Part 3 of this document. Add `@Column`, `@NotNull`, and enums.
CONFIG: Each service points at its own database in `application.yml`. Set `spring.jpa.hibernate.ddl-auto=update` for now.
REPOSITORIES: Create a Spring Data repository interface per entity.
VERIFY: Boot each service, then open a DB client (pgAdmin / DBeaver / Mongo Compass) and confirm the tables/collections were created.
CHECK: user-service owner double-checks: **there is no `semester` column anywhere.**
DELIVERABLE: All 6 databases have real tables/collections, created from real entity code.

Day 3 — auth-service End-to-End (priority build)
OTP REQUEST: `POST /api/auth/otp/request` — generate a 6-digit OTP, SHA-256 hash it, store with `expires_at = now + 10 min`, log the plain OTP to console (real email comes Day 5).
OTP VERIFY: `POST /api/auth/otp/verify` — hash the submitted OTP, compare, check expiry, check `attempts < 5`, mark used, issue JWT.
JWT: `JwtUtil.java` — generate with claims `{userId, email, role, campusId}`, HMAC-SHA256, 24-hour expiry.
REDIS: Store the attempt counter in Redis with TTL so brute-force lockout survives restarts.

Day 3 — auth-service End-to-End (priority build)
AUDIT: Write an `audit_logs` row on every LOGIN and OTP_FAILED.
EVERYONE ELSE: Build one real Create + one real Read endpoint on your own service today, so you have something to protect tomorrow.
TEST: In Swagger — request OTP, copy it from the console, verify, receive JWT. Try a wrong OTP 5 times → locked.
DELIVERABLE: Full OTP → JWT flow works via Swagger.

Day 4 — Shared JWT Validation Across All 6 Services
SHARED FILTER: auth-service owner writes a small `JwtAuthFilter` + `SecurityConfig` that validates a token's signature and expiry **locally** (no network call to auth-service per request), and posts it in the team chat as a copy-paste snippet.
ADOPT: All 5 other owners drop it into their service, set the same `JWT_SECRET` env var, and protect at least one endpoint.
INTERNAL API KEY: Add a shared `X-Internal-Key` header check for `/internal/**` endpoints — these are service-to-service only, no JWT.
CRUD: gatepass, user, and campus owners finish first real Create + Read endpoints for their main entities.
TEST: Every service — call a protected endpoint with no token → 401. With a valid token → 200. With an expired token → 401.
DELIVERABLE: All 6 services enforce the same JWT. Internal endpoints are key-protected.

Day 5 — Real Email + MILESTONE M1
SMTP: auth-service owner adds `spring.mail.*` config (Gmail SMTP, port 587, STARTTLS, **App Password not main password**).
EMAIL SERVICE: `EmailService.sendOtp(email, otp)` with a simple HTML template. Switch from console logging to real sending.
SECRETS: Move `JWT_SECRET`, `MAIL_USERNAME`, `MAIL_PASSWORD` into a `.env` file. Commit `.env.example` only — **never the real** `.env`.
TEAM SYNC (45 min): Every member demos their service's Swagger page live.

Day 5 — Real Email + MILESTONE M1
DELIVERABLE — M1: Every service has real entities, ≥2 working endpoints, Swagger docs, and JWT protection. A real OTP email lands in a real inbox.

Week 2 — Core Backend Pipeline (Days 6–12)

Day 6 — Visitor Request Flow + Profiles
GATEPASS: `POST /api/gatepass/visitor-requests` — validate date range, require OTP verification before accepting, save with `status = PENDING`.
GATEPASS: `PUT /api/gatepass/visitor-requests/{id}/approve` — create a `gate_passes` row with `pass_type = DAILY` or `EVENT`, `status = PENDING`, no QR yet.
USER: Finish student + faculty CRUD, department list endpoint, `POST /api/users/documents` with a placeholder S3 key for now.
CAMPUS: Finish campus + gate CRUD. Seed CDAC Mumbai with Main Gate and Back Gate.
GUARD: Build `entry_logs` document + repository with the 4 indexes from Part 3.5.
QR: Build `qr_records` + `generation_jobs` entities and repositories.
TEST: Via Swagger — submit a visitor request, approve it, confirm a PENDING pass row exists in the database.
DELIVERABLE: Visitor request → approval → PENDING pass, provable in the DB.

Day 7 — RabbitMQ Wiring (producer + consumer handshake)
CONFIG: Both gatepass-service and qr-service add `spring-boot-starter-amqp`. Declare queue `pass.generate` and a dead-letter queue `pass.generate.dlq`.
PRODUCER: gatepass-service — on approval, `rabbitTemplate.convertAndSend("pass.generate", payload)` where payload is `{passId, holderName, campusId, validFrom, validTo}`.
CONSUMER: qr-service — `@RabbitListener(queues="pass.generate")` that today only logs the received payload and writes a `generation_jobs` row with status QUEUED.
TEST: Approve a request in Swagger → watch the qr-service console print the payload → check RabbitMQ UI shows the message consumed → check `generation_jobs` has a row.

Day 7 — RabbitMQ Wiring (producer + consumer handshake)

DELIVERABLE: The message actually crosses from one service to another. This handshake is the foundation of Days 8–11.

Day 8 — QR Service: Token + QR + PDF + S3 (the big build day)

TOKEN: `TokenService.generate(passId, campusId, expiry)` — AES-256 encrypt, Base64 encode. Also produce a SHA-256 `token_hash` for lookup.

QR: `QRCodeService.generate(token)` — ZXing `BarcodeFormat.QR_CODE`, 300×300 PNG, return byte array.

PDF: `PdfService.generate(pass, qrBytes)` — iText/OpenPDF document with QR image, holder name, campus, validity dates.

S3: `S3Service.upload(key, bytes, contentType)`. If the AWS account is not ready, use **LocalStack** today — do not let AWS signup block this day.

WIRE IT: Inside the `@RabbitListener` — token → QR → PDF → upload both → save `qr_records` → job DONE → call `PUT /api/gatepass/internal/passes/{id}/activate`.

GATEPASS: Implement that internal activate endpoint — store both S3 keys, set `status = ACTIVE`.

TEST: Approve a request → wait ~2 seconds → the pass is ACTIVE, `qr_s3_key` and `pdf_s3_key` are populated, and both files exist in S3/LocalStack.

DELIVERABLE: Full async pipeline working end-to-end, no frontend involved.

Day 9 — Guard Service: Scan Logic (GREEN / RED)

QR INTERNAL: `POST /api/qr/internal/decrypt` — decrypt the token, look up `qr_records` by `token_hash`, return `{passId, holderUserId, holderName, passType, eventId, validFrom, validTo, isActive}` or 404.

GUARD SCAN: `POST /api/guard/scan` — call qr-service to decrypt, then check: token valid? `is_active`? today within `valid_from` … `valid_to`?

RESULT: GREEN with holder name, or RED with a specific `deny_reason` (INVALID_TOKEN / REVOKED / EXPIRED / NOT_YET_VALID).

LOG: Write an `entry_logs` document for **both** outcomes — denied attempts are security data, never discard them.

Day 9 — Guard Service: Scan Logic (GREEN / RED)

TEST: Scan a real generated token → GREEN. Scan gibberish → RED INVALID_TOKEN. Manually expire a pass in the DB → RED EXPIRED. Check MongoDB has 3 documents.

DELIVERABLE: The core gate scan works. This is the heart of the whole product.

Day 10 — Events + MILESTONE M2

EVENTS: gatepass-service — `events` table, `POST /api/gatepass/events`, `GET /api/gatepass/events`. Add `pass_type` and `event_id` handling to pass creation.

EVENT PASS: Approving a request tied to an event produces `pass_type = EVENT`, `valid_from` / `valid_to` copied from the event, `event_id` set.

GUARD: If the scanned pass is an EVENT pass, log `event_id` on the entry and return GREEN "Welcome to [Event]".

TEAM SYNC: Demo the **entire backend pipeline** service by service through Swagger.

DELIVERABLE — M2: visitor request → approve → RabbitMQ → QR + PDF on S3 → pass ACTIVE → guard scan → GREEN, with an event pass variant. Fully provable without any UI.

Day 11 — The Bulk Engine

PARSE: Apache POI reads the uploaded `.xlsx`. Minimum columns for an event batch: `name, email, phone, purpose`. Visit dates come from the **event**, not per row.

VALIDATE: Per row — valid email format? duplicate within this sheet? already has a pass for this event? on the campus blocklist? Collect errors as `row 34: invalid email`.

SUMMARY: Return `{batchId, totalRows, validRows, invalidRows}` fast (~2 seconds). Status = AWAITING_CONFIRM. Write the error report Excel to S3.

RESOLVE: `POST /bulk-upload/{batchId}/confirm` — for each valid row, call `GET /api/auth/internal/users/by-email`. Exists → reuse identity. New → create lightweight VISITOR identity. (Part 6.2.)

PUBLISH: Create N pass rows, publish N jobs to `pass.generate`, set batch status = PROCESSING, return immediately.

AUTH: Build the `by-email` internal lookup endpoint the bulk engine depends on.

TEST: Upload a 10-row sheet: 3 emails that already exist as students, 5 new, 1 duplicate, 1 malformed. Expect 8 valid / 2 errors, 5 new identities created, 8 passes generated, zero duplicate accounts.

Day 11 — The Bulk Engine
DELIVERABLE: Mixed-attendee resolution provably correct on a real Excel file.

Day 12 — Behavior 2, AMBER, and Attendance Aggregation
INTERNAL: gatepass-service — `GET /internal/passes/{userId}/active-event` returns the event id if that person has an event running today, else null.
BEHAVIOR 2: guard-service — if the scanned pass is DAILY, call that endpoint. If an event is running, log the entry with that `event_id` and return GREEN "Welcome to [Event]". Otherwise normal GREEN "Welcome".
AMBER: Define and implement the amber cases — pass valid but expires today, or campus config `approval_required` demands host confirmation. Return AMBER with a message, still log it.
ATTENDANCE: guard-service `GET /internal/attendance/{eventId}` aggregates distinct holders per day from MongoDB. gatepass-service `GET /events/{id}/attendance` combines it with the registered count.
TEST: Create a student with a DAILY pass, enrol them in an event running today, scan their **daily** QR → GREEN attributed to the event. Verify the attendance count incremented.
DELIVERABLE: The two-QR problem is solved and demonstrable. Attendance numbers are correct.

Week 3 — Frontend (Days 13–17)

Day 13 — React Setup + Auth Screens (whole-team kickoff)
SETUP: `npm create vite@latest frontend -- --template react`. Install axios, react-router-dom, tailwindcss. One person does this and pushes it.
SHARED (30-min team huddle): Agree the shell — `App.jsx` routes, `AuthContext.jsx`, `ProtectedRoute.jsx`, Navbar, Sidebar, Toast. Auth owner implements it; everyone reviews the PR together.
AXIOS: `api/client.js` — base instance with an interceptor that attaches `Authorization: Bearer <jwt>` to every request and redirects to login on 401.
SCREEN 1 + 2: Email entry → OTP verify with 6 boxes, 10-minute countdown, resend link, attempts-left warning. On success store JWT and redirect by role.
FOLDERS: Every other owner creates `frontend/src/<their-service>/` with one placeholder page wired into routing.

Day 13 — React Setup + Auth Screens (whole-team kickoff)
TEST: Log in end-to-end with a real OTP email → land on a role-specific placeholder page → refresh the browser → still logged in.
DELIVERABLE: Real login works in the browser. Every member has a routed page to build into.

Day 14 — Core Screens, Part 1
USER (3, 4, 5): Student profile view/edit (**no semester field**), faculty profile, student directory with department filter chips and pagination.
GATEPASS (6, 7): Public visitor registration form with the OTP step inline, and the admin approval queue with Approve/Reject and a reject-reason modal.
CAMPUS (16): Campus list/edit, gate management table, config toggles.
PATTERN: Every screen must handle three states — loading spinner, error message, and empty state. Agree the look of these once and reuse.
TEST: Each owner clicks through their own screens against their real running backend.
DELIVERABLE: Half the UI is real and calling real APIs.

Day 15 — Core Screens, Part 2 + MILESTONE M3
GUARD (13): Scanner screen — camera QR capture with a manual-token fallback input. Full-screen GREEN / RED / AMBER result card showing holder name and message, auto-reset after 5 seconds. Make the colour fill the whole screen — it must be readable at arm's length on a tablet.
QR (14): Pass download page — large QR image, validity dates, Download PDF button.
GATEPASS (8): My Pass screen — must correctly render **two passes side by side** when a student holds both DAILY and EVENT.
TEAM SYNC — M3: Run one complete journey through the actual UI: visitor registers → admin approves → email arrives with PDF → guard scans → GREEN on screen.
DELIVERABLE — M3: A full user journey works through real screens, not Swagger.

Day 16 — Bulk + Attendance Screens (the demo centrepiece)
GATEPASS (9): Bulk upload — drag-drop Excel, type selector, event picker, then the **"580 valid, 20 errors"** summary panel with an error-report download link and a Confirm button.

Day 16 — Bulk + Attendance Screens (the demo centrepiece)
GATEPASS (10): Bulk progress — poll `/jobs/batch/{batchId}/progress` every 2 seconds, show "412 of 580 generated".
GATEPASS (11, 12): Event create form, and the attendance dashboard — Registered / Attended Day 1 / Day 2 / Never showed cards, a per-day bar chart, attendee search, Export CSV.
GUARD (15): Guard log table with date and result filters, colour-coded result badges.
TEST: Upload a 50-row sheet in the browser → see the summary → confirm → watch the progress bar move → open the attendance dashboard.
DELIVERABLE: The strongest demo screens are working.

Day 17 — Welcome Email + Notifications
QUEUE: Add a `notification.send` RabbitMQ queue. qr-service publishes to it after a pass goes ACTIVE.
EMAIL: Consumer builds the welcome email — subject "Your gate pass for [Event] at CDAC Mumbai", body with name/dates/gate, and the **PDF pass attached** (fetched from S3). One email, not two.
BULK EMAIL: Confirm 50 emails send without blocking, and that one bad address does not stop the rest.
TEST: Approve a request → within seconds a real email arrives with a real PDF attachment that opens and shows a scannable QR. Scan that exact PDF's QR at the guard screen → GREEN.
DELIVERABLE: The complete real-world loop closes — register, receive pass by email, walk to gate, scan, enter.

Week 4 — Hardening (Days 18–20)

Day 18 — Error Handling + Validation Everywhere
GLOBAL HANDLER: Every service adds `@RestControllerAdvice` returning a consistent shape: `{success, message, data, errors}`. Map exceptions to proper codes — 400 validation, 401 auth, 403 role, 404 missing, 409 conflict. Never leak a stack trace.
VALIDATION: Add `@Valid`, `@NotBlank`, `@Email`, `@Future` to every request DTO across all services.
FRONTEND: Every form shows field-level error messages. Every API failure shows a toast, never a blank screen.

Day 18 — Error Handling + Validation Everywhere
EDGE CASES: Test a 50-row Excel with deliberately broken rows. Test AMBER paths. Test scanning a revoked pass. Test an OTP after 11 minutes.
DELIVERABLE: No unhandled 500 anywhere on expected bad input.

Day 19 — Cross-Team Regression Testing
PAIR UP: Test **someone else's** service, never your own. Rotate: 1↔4, 2↔5, 3↔6. Bugs found by the author are the ones that were never going to be found.
SCENARIO 1: Single visitor — register → OTP → approve → email → scan → GREEN. Scan again → still GREEN (entry-only allows repeat entries — verify this is not treated as an error).
SCENARIO 2: Bulk student import — 20 rows → all DAILY passes, no end date.
SCENARIO 3: Mixed event batch — existing students reused, new visitors created, zero duplicates.
SCENARIO 4: Behavior 2 — student scans DAILY QR during their event → attendance still credited.
SCENARIO 5: All three colours — GREEN, RED (each of the 4 deny reasons), AMBER.
SCENARIO 6: RBAC — a STUDENT token calling an ADMIN endpoint → 403 everywhere.
LOG: Every bug becomes a GitHub Issue with a severity label. **P1 = breaks the demo. P2 = cosmetic.**
DELIVERABLE: Every bug has an issue number, an owner, and a severity.

Day 20 — Bug Fix Day + MILESTONE M4
FIX: All P1 issues from Day 19. P2 only if P1 is clear.
INDEXES: Add PostgreSQL indexes on `gate_passes(holder_user_id)`, `gate_passes(status)`, `visitor_requests(status)`, `otp_verifications(email)`. Verify the MongoDB compound index is actually being used with `.explain()`.
REVIEW: Walk all 16 screens manually. Look for broken layouts, missing empty states, untranslated raw JSON errors.
TEAM SYNC — M4: Re-run all 6 scenarios from Day 19.
DELIVERABLE — M4: Every core journey works with zero manual workarounds.

Week 5 — Deploy & Demo (Days 21–25)

Day 21 — Seed Data + Dockerfiles

DOCKERFILES: Multi-stage Dockerfile per service — `maven:3.9-eclipse-temurin-17` to build, `eclipse-temurin:17-jre` to run. Frontend: `node:20` build + `nginx:alpine` serve.

COMPOSE: Uncomment all 6 service blocks in `docker-compose.yml`. Add `depends_on`, health checks, and env vars from `.env`.

SEED: `DataSeeder` per service, running only when the DB is empty. Seed: CDAC Mumbai campus, Main Gate + Back Gate, all 6 departments, `admin@cdac.in`, `guard@cdac.in`, ~15 students with DAILY passes, 1 event "AI Summit" with ~30 attendees and a few days of realistic scan history so the attendance dashboard is not empty.

TEST: `docker-compose down -v && docker-compose up --build` from scratch. All 10 containers healthy. Log in and see seeded data.

DELIVERABLE: One command starts the whole system, fully seeded and demo-ready.

Day 22 — AWS EC2 Deployment

EC2: Launch a t3.medium Ubuntu instance (t2.micro is too small for 10 containers). Security group: open 22 (SSH, your IP only), 80, 3000. **Do not expose the database ports publicly.**

INSTALL: SSH in. Install Docker and Docker Compose on the instance.

DEPLOY: Clone the repo onto the instance. Create the real `.env` there by hand — secrets live on the server, never in Git. `docker-compose up -d --build`.

VERIFY: `docker ps` shows 10 healthy containers. Open the frontend at the instance's public IP.

GOTCHAS: The frontend's API base URL must point at the public IP, not localhost. CORS on each service must allow that origin. Budget real time for exactly these two problems.

DELIVERABLE: The system is live and reachable from any browser.

Day 23 — Real S3 + Live Smoke Test + MILESTONE M5

S3: Create the real `perimity-bucket`. Create an IAM user with **bucket-scoped** permissions only. Put the credentials in the server `.env`. Switch off LocalStack.

CORS: Configure bucket CORS so pre-signed browser uploads work.

Day 23 — Real S3 + Live Smoke Test + MILESTONE M5
SMOKE TEST — on the deployed system, not localhost: register a visitor → approve → email arrives → open the PDF on a phone → scan that phone screen at the guard screen on a laptop → GREEN.
BULK ON PROD: Upload a 50-row sheet on the live system and confirm all passes generate.
DELIVERABLE — M5: The complete demo journey works against the live deployment with real S3.

Day 24 — Documentation + Slides
README: Prerequisites, clone + run commands, `.env.example` explanation, seeded demo credentials, service/port table, ASCII architecture diagram.
DIAGRAM: One clean architecture diagram — 6 services, 2 database types, RabbitMQ, Redis, S3, React. Everyone must be able to point at their own box and explain it.
SLIDES: Problem → architecture → the 3 core technical pieces (Part 6) → live demo → learnings. Keep it under 12 slides.
ASSIGN: Who presents which section. Natural split: each owner presents their own service in the architecture slide.
DELIVERABLE: README and slides complete and committed.

Day 25 — Dress Rehearsal + Code Freeze + MILESTONE M6
REHEARSE: Run the full Part 9 demo script end-to-end, timed, **three times**. Every member knows their cue.
CONTINGENCY: Record a 3-minute screen capture of the working demo as backup if the venue's network fails. Have the seeded local Docker setup ready as a second fallback.
VIVA DRILL: Mock viva — each member answers 3 questions about their own service, unprompted, without notes.
FREEZE: **No new features from this point.** Critical bug fixes only, and only with a second person reviewing.
DELIVERABLE — M6: Demo rehearsed 3+ times. Every member can explain their service in 60 seconds. Backup video recorded.

Days 26–30 — Buffer (do not schedule features here)

Reserve this for slippage. Days 18–20 are statistically where student projects overrun. If genuinely unused, in priority order: (1) domain name + HTTPS, (2) load-test the real 600-row bulk scenario, (3) UI polish, (4) prepare Eureka/scaling as a *talking point* — not as something to build.

Part 8 — Project Folder Structure

```
perimity/
├── docker-compose.yml              ← one command starts everything
├── .env.example                    ← template; real .env is never committed
├── README.md
├── .github/
│   ├── workflows/docker-build.yml  ← auto-builds every service image on push
│   └── PULL_REQUEST_TEMPLATE.md
├── docs/
│   ├── Perimity_SRS_v1.1.md
│   ├── Perimity_Database_Design.md
│   └── Perimity_Event_Bulk_Design.md
│
├── auth-service/
│   ├── Dockerfile
│   └── src/main/java/in/cdac/perimity/auth/
│       ├── controller/   (AuthController, AuditController)
│       ├── service/      (OtpService, JwtService, EmailService, AuditService)
│       ├── repository/   (UserRepository, OtpRepository, AuditLogRepository)
│       ├── entity/       (User, OtpVerification, AuditLog)
│       ├── dto/          (OtpRequest, OtpVerifyRequest, AuthResponse)
│       └── security/     (JwtUtil, JwtAuthFilter, SecurityConfig)
│
├── user-service/
│   └── src/main/java/in/cdac/perimity/user/
│       ├── controller/   (StudentController, FacultyController, DepartmentController, DocumentControl
│       ├── service/      (StudentService, FacultyService, S3PresignService)
│       ├── repository/   (StudentProfileRepository, FacultyProfileRepository, DepartmentRepository, D
│       ├── entity/       (StudentProfile, FacultyProfile, Department, Document)
│       └── dto/
│
├── gatepass-service/
│   └── src/main/java/in/cdac/perimity/gatepass/
│       ├── controller/   (VisitorRequestController, PassController, EventController, BulkUploadContro
│       ├── service/      (PassService, EventService, BulkUploadService, IdentityResolver)  ← Identity
│       ├── repository/   (VisitorRequestRepository, GatePassRepository, EventRepository, BatchReposit
```

```
│       ├── entity/       (VisitorRequest, GatePass, Event, BulkUploadBatch)
│       ├── excel/        (ExcelParser, RowValidator, ErrorReportWriter)
│       ├── messaging/    (PassGenerationProducer)
│       └── dto/
│
├── campus-service/
│   └── src/main/java/in/cdac/perimity/campus/
│       ├── controller/   (CampusController, GateController, ConfigController)
│       ├── service/      (CampusService, ConfigService)
│       ├── repository/   (CampusRepository, GateRepository, ConfigRepository)
│       └── entity/       (Campus, CampusGate, CampusConfig)
│
├── guard-service/
│   └── src/main/java/in/cdac/perimity/guard/
│       ├── controller/   (ScanController, EntryLogController)
│       ├── service/      (ScanService, AttendanceService)   ← ScanService = Part 6.3 decision tree
│       ├── repository/   (EntryLogRepository)               ← Spring Data MongoDB
│       ├── document/     (EntryLog)
│       ├── client/       (QrServiceClient, GatePassServiceClient)
│       └── dto/          (ScanRequest, ScanResult)
│
├── qr-service/
│   └── src/main/java/in/cdac/perimity/qr/
│       ├── controller/   (QrController, JobController)
│       ├── service/      (TokenService, QRCodeService, PdfService, S3Service)
│       ├── repository/   (QrRecordRepository, GenerationJobRepository)
│       ├── entity/       (QrRecord, GenerationJob)
│       ├── messaging/    (PassGenerationConsumer, NotificationProducer)  ← the @RabbitListener
│       └── config/       (RabbitConfig, AwsConfig)
│
└── frontend/
    ├── Dockerfile
    └── src/
        ├── api/          (client.js, authApi.js, userApi.js, gatepassApi.js, guardApi.js, qrApi.js, c
        ├── shared/       (Navbar, Sidebar, ProtectedRoute, Toast, LoadingSpinner, EmptyState, AuthCor
        ├── auth/         (EmailEntry, OtpVerify)
        ├── users/        (StudentProfile, FacultyProfile, StudentDirectory)
        ├── gatepass/     (VisitorRegistration, ApprovalQueue, MyPass, BulkUpload, BulkProgress, Event
        ├── campus/       (CampusAdmin, GateManagement)
        ├── guard/        (GuardScanner, GuardLog)
        ├── qr/           (PassDownload)
        └── utils/        (jwtUtils.js, dateUtils.js, constants.js)
```

Part 9 — Milestones

Milestone	Day	What Must Be True	How to Verify
M1	Day 5	All 6 services boot with real entities, Swagger, and JWT protection. Real OTP email delivers.	Open all 6 Swagger pages. Request an OTP, receive a real email, verify, get a JWT.
M2	Day 10	Full backend pipeline: request → approve → RabbitMQ → QR + PDF on S3 → ACTIVE → scan → GREEN.	Swagger only, no UI. Check S3 for both files. Check MongoDB for the entry log.
M3	Day 15	One complete user journey works through real UI screens.	Browser: register → approve → scan → green card fills the screen.
M4	Day 20	All 6 test scenarios pass. All P1 bugs closed.	Re-run the Day 19 scenario list. Zero manual workarounds.
M5	Day 23	Live on EC2 with real S3. Demo journey works against the deployed system.	Open the public IP on a phone. Complete the journey there.
M6	Day 25	Demo rehearsed 3×. Every member explains their service in 60 seconds. Backup video exists.	Mock viva with no notes. Timed rehearsal completes without failure.

Part 10 — Demo Flow Script (10 minutes)

Assign one member per section. Rehearse three times on Day 25.

1. INTRO (30 sec) — "Every CDAC gate still runs on a paper register. A visitor writes their name, a guard squints at it, and nobody can ever search it again. Perimity replaces that register with QR passes that are time-bound, forgery-proof, and searchable — and it scales from one visitor to a 600-person event."

2. VISITOR REGISTERS (1.5 min) — Open the public registration form. Fill in a visitor, submit. OTP arrives in the inbox on screen; enter it. "No password, ever. A one-time visitor never creates an account — email is the only identity they need."

3. ADMIN APPROVES (1 min) — Log in as `admin@cdac.in` (via OTP, on screen). Open the approval queue, approve the request. "That response came back instantly — because nothing slow happened yet. The pass generation was pushed onto a RabbitMQ queue."

4. THE EMAIL ARRIVES (1 min) — Switch to the inbox. A welcome email has arrived with the PDF pass attached. Open it — QR, name, validity dates. “Generated in the background: token encrypted, QR rendered, PDF composed, uploaded to S3, emailed. The admin never waited for any of it.”

5. GUARD SCANS — GREEN (1 min) — Open the guard scanner on a second screen. Scan the QR straight off the phone. Full-screen green, visitor name. “One scan, one colour. The guard makes no decisions.”

6. GUARD SCANS — RED (45 sec) — Scan a revoked or expired pass. Full-screen red with the reason. “Denied attempts are logged too — that’s security data a paper register throws away.”

7. THE BULK STORY (2.5 min) — “Now the part a paper register could never survive.” Open bulk upload, drop in a 600-row AI Summit sheet. Summary appears: “580 valid, 20 errors” with a downloadable error report. Confirm. Progress bar starts moving. “102 of those 600 are already CDAC students. The faculty doesn’t know which ones — and doesn’t need to. The engine matches every row by email: existing people get an event pass on their existing identity, brand-new people get a lightweight visitor identity. Zero duplicates.”

8. BEHAVIOR 2 (1 min) — Take a student who holds both a DAILY and an EVENT pass. Scan their **daily** QR. Green — “Welcome to AI Summit.” “They scanned the wrong QR out of habit. The system attributed the entry to the event anyway, so the organizer’s attendance stays accurate. The guard never had to know.”

9. THE PAYOFF (1 min) — Open the attendance dashboard. Registered 600, Attended Day 1 543, Day 2 478, Never showed 41. Export CSV. “This is the strongest argument for the whole system. No register in the world produces this.”

10. ARCHITECTURE (45 sec) — Show the diagram. Each member names their service in one sentence. Point at RabbitMQ: “This is why the 600-row upload didn’t time out.” Point at MongoDB: “This is why millions of scans stay fast.”

Part 11 — Viva Preparation

Every member must answer questions about **their own service** plus these architecture questions.

Q: Why microservices and not a monolith for a student project?
Team ownership: 6 people, 6 services — each person owns a service end-to-end and can build, run, and deploy it without waiting on anyone.
Independent scaling: the guard scan endpoint is high-frequency during event mornings; it can scale without scaling profile management.
Failure isolation: if the QR generation service crashes mid-batch, visitors can still register and guards can still scan existing passes.

Q: Why microservices and not a monolith for a student project?

Honest caveat: for a single-campus deployment a monolith would be simpler. We chose microservices deliberately for the ownership model and to demonstrate distributed-system competence — and we can defend where it cost us (cross-service calls, no cross-DB joins).

Q: Why did you not use Eureka / Spring Cloud service discovery?

Eureka solves dynamic discovery — many instances per service, appearing and disappearing, needing load balancing.

We run exactly one instance per service under Docker Compose. Docker's built-in DNS already resolves `http://qr-service:8080` by container name.

Adding a registry would mean one more service to run, configure, monitor, and fail — with zero benefit at this scale.

The moment we scale to multiple instances per service or move to Kubernetes, service discovery becomes necessary. Choosing not to add it now is a scoping decision, not an oversight.

Q: Why RabbitMQ? Why not just call the QR service directly over HTTP?

Time: 600 passes means 600 token generations, 600 QR renders, 600 PDFs, 600 S3 uploads, 600 emails. Synchronously that is minutes — the browser times out.

Decoupling: the gate pass service should not care whether the QR service is even running. It publishes the job and returns.

Resilience: if the QR service is down, jobs queue in RabbitMQ and process when it restarts. Nothing is lost. With a direct HTTP call, the approval would simply fail.

Independent failure: one bad row fails one job. The other 579 are unaffected. A synchronous loop would abort the whole batch.

Q: Why two databases — PostgreSQL and MongoDB?

Gate passes and approvals need transactions, foreign keys, and consistency — that is PostgreSQL's job.

Gate scans are append-only, join-free, self-contained events arriving at high volume — millions of rows over years. That is MongoDB's job.

The scan document needs no relationships: pass id, guard, gate, timestamp, result. Forcing it into a relational schema adds JOIN cost for no gain.

Q: Why two databases — PostgreSQL and MongoDB?

We index MongoDB on `campus_id + scanned_at` as a compound index, which covers nearly every attendance and log query in a single index scan.

Q: Why passwordless? Isn't OTP less secure than a password?

There is no password to leak, reuse across sites, forget, or phish. The credential is a 10-minute, single-use, SHA-256-hashed code.

The primary user is a one-time visitor. Forcing account creation for a single gate entry would kill adoption — most people would abandon the form.

Brute force is bounded: 5 attempts, then locked. Expiry is 10 minutes. Codes are single-use and never stored in plain text.

Honest limitation: OTP security depends on the user's email account being secure, and email delivery adds latency. In production we would add rate-limiting per IP and consider TOTP for staff accounts.

Q: How does the system handle a person who is both a student and an event attendee?

We separate identity from pass. One identity per person, keyed by email, forever. A pass is permission to enter for a purpose and a time window.

That person holds two active passes simultaneously: a DAILY pass with no end date, and an EVENT pass valid only for the event dates. Like a company ID badge plus a conference lanyard.

The bulk engine matches by email, recognizes them as existing, and issues only the event pass — it never creates a duplicate account.

At the gate, if they scan the daily QR during the event, Behavior 2 auto-attributes the entry to the running event so the organizer's attendance stays correct.

Q: Explain your bulk upload flow and why it is split into two phases.

Phase 1 (fast, synchronous, ~2 seconds): parse the Excel, validate every row, check duplicates and blocklist, return "580 valid, 20 errors" plus a downloadable error report. The faculty is still watching the screen for this.

Phase 2 (slow, asynchronous): after the faculty clicks Confirm, we create identities and publish one RabbitMQ job per pass, then return immediately. The faculty can close the laptop.

The split exists because validation must be interactive — the faculty needs to see errors and decide — but generation must not be. Mixing them would either time out or hide errors until it was too late to fix them.

Q: Explain your bulk upload flow and why it is split into two phases.

Bad rows never block the batch. The faculty fixes only the flagged rows and re-uploads just those.

Q: What happens if the QR service crashes halfway through a 600-pass batch?

Unprocessed jobs stay in the RabbitMQ queue — they are not lost, because we acknowledge a message only after the job completes.

When the service restarts, the `@RabbitListener` resumes consuming from where it stopped.

Passes already generated are ACTIVE. Passes not yet generated remain PENDING with a QUEUED job row, so the state is always recoverable and auditable.

Individually failing jobs retry up to 3 times, then move to a dead-letter queue and are marked FAILED with the error message for manual escalation.

Q: How is the QR code secure? Can someone forge one?

The QR does not encode a pass ID or a name — it encodes an AES-256 encrypted token containing pass_id, campus_id, and expiry.

Without the server-side key, an attacker cannot construct a token that decrypts to anything valid.

Even a correctly decrypted token is then checked against the database: does this token hash exist, is `is_active` true, is today within the validity window?

A screenshot of someone else's valid QR would work — that is a deliberate, documented tradeoff, exactly as a paper pass could be handed over. Mitigation in production would be to display the holder's photo on the guard's green screen so the guard can match face to pass.

Q: Why is it entry-only? Why no exit scan?

The system's stated goal is to replace the paper register, and that register only ever recorded entry.

Exit scanning doubles gate friction and halves throughput at peak times — a real cost for a marginal data gain.

Repeat entries are all logged as separate rows, exactly as a register would have multiple lines for the same person.

For a multi-day event, each day's first scan is that day's attendance, which is what an organizer actually needs.

Q: What are the roles and what can each do?
VISITOR: register a visit, verify by OTP, view and download their own pass. Nothing else.
STUDENT / FACULTY: hold a DAILY pass, view their own profile and passes. Faculty additionally can create events and run bulk uploads.
GUARD: the scan endpoint and their own gate's log. Cannot create passes, cannot see profiles.
ADMIN: campus, gates, config, departments, approvals, audit logs, all passes for their campus. Scoped by `campus_id` — an admin of one campus cannot see another's data.

Q: What are the known limitations of your system?
Single instance per service — no high availability. If a container dies, that capability is down until it restarts.
No API gateway, so the frontend calls six ports directly and CORS must be configured per service.
A QR screenshot can be shared; we mitigate by showing the holder's name and photo on the green screen, not by preventing it technically.
Cross-service data requires an API call, so some screens make two round trips where a monolith would use one JOIN.
Stating these clearly is deliberate — every architecture has tradeoffs, and knowing yours is the point.

Appendix — Rules Recap (print this and stick it on the wall)

1. **No `Semester` field in any UI. Ever.**
2. Passwordless everywhere — email + OTP, including admins.
3. Files go to S3; the database stores only the S3 key.
4. bcrypt for passwords, SHA-256 for OTPs, AES-256 for QR tokens. Nothing in plain text.
5. No service reads another service's database. API calls only.
6. Every endpoint gets a Swagger `@Operation` summary before it counts as done.
7. Never push to `main`. Branch → PR → 1 review → merge.
8. Real secrets live in `.env`, which is never committed. Only `.env.example` is.